Vishwakarma Institute of Technology, Pune-411037, Maharashtra, India
CS3111 Web Technologies · Computer Engineering · AY 2026-27

PPT Content Guide

PS Laser Web Platform — slide-by-slide technical content

Rahul Diwate
Rivan Shetty
Gaurav Shende
Vishal Sharma
Gandharv Shinde

21 October 2026

How to use this document

This guide gives the content for every slide in the order you specified: **Information, Introduction, Problem Statement, Aim, Objectives, Literature Review, Technologies, System Architecture (with DFD), Methodology (all modules: description and workflow), Results, Testing (with the software development model), Advantages and Disadvantages, Conclusion.** Each slide has two parts: short text that belongs ON the slide, and technical notes you SAY. Everything is written from the system side (components, requests, data flows), not from a user's point of view.

The slide numbering below gives 55 slides. A shorter 15-minute path is listed in section 12.6. Image files for the slides (DFDs, architecture, Unit V layered diagram, screenshots) are separate PNGs in `docs/ppt/` so you can insert them at full resolution. Evidence tags: **[V]** verified by running or measuring on the stated date, **[S]** read from source code or project documents, **[P]** proposed or a design decision to confirm.

Two things to confirm before presenting: (1) the course faculty on your Unit V slides is Dr. Rahul B. Diwate, and the paper lists "Rahul Diwate" as first author. If that is the same person, write "Dr. Rahul B. Diwate" with designation on the Information slide. (2) The development model on Slide 46 is the model the project evidence supports (iterative-incremental, contract-first). No formal process record exists, so confirm it with your team.

Naming standard (use these names on every slide)

Table A. Uniform terminology

Concept	Use this term	Do not use
Whole system	PS Laser Web Platform	website, app, software (alone)
Next.js application (port 3001)	Next.js Web Application	frontend, main site
React portal (5173)	React Portal Application	customer app
PHP estimator (8000)	PHP Estimator Application	PHP site
Laravel back office (8001)	Laravel Admin Application	admin panel (alone)
Servlet/JSP (8080)	Servlet/JSP Desk Application	Java site
Spring Boot (8081)	Spring Boot Core API	backend (alone)
Node.js Express (4000)	Node.js Enquiry API	upload server
MySQL/MariaDB database pslaser	MySQL database (pslaser)	DB, SQL server
MongoDB (27017)	MongoDB activity log	NoSQL DB
Browser user	Client	customer, visitor, end user
Admin or staff role	Staff user	owner, operator
Price calculation result	estimate (INR, indicative)	quotation, final price
Saved estimate record	quote (table quotes)	bill, invoice
Submitted request	enquiry (table enquiries)	lead, ticket
Job record	work order (table work_orders)	order, job card
One row of job history	status event (table work_order_events)	log entry
Seven job stages	received, cutting, bending, welding, coating, inspection, dispatched	steps, phases
Front-end to API middle layer	BFF route handler (Backend-for-Frontend)	proxy, middleware
In-process message	domain event	notification

Part 1 — Information

Slide 1 — Title

On the slide

- PS Laser Web Platform: A Multi-Stack Web System for Cost Estimation, Enquiry Capture, Order Tracking and Work-Order Management in a Laser-Cutting Job Shop
- CS3111 Web Technologies · Department of Computer Engineering · AY 2026-27

- Vishwakarma Institute of Technology, Pune-411037
- Team: Rahul Diwate, Rivan Shetty, Gaurav Shende, Vishal Sharma, Gandharv Shinde
- Guide / Course faculty: Dr. Rahul B. Diwate (confirm designation and role)

Explain (technical notes for the speaker)

- State the title once, slowly. Say that the platform is one system built from seven cooperating services over one shared database, and that each service maps to a unit of the CS3111 syllabus.

Visual / file: Use the Unit V deck frame (rounded border, orange title band, date in the footer).

Slide 2 — Project information

On the slide

- Domain: sheet-metal and CNC laser-cutting job shop (PS Laser, Velu, Bhor, Pune)
- System type: multi-service web platform, 7 applications + 2 data stores
- Syllabus coverage: Units I–VI (PHP/Laravel, React/Redux, Servlet/JSP, Spring Boot, Advanced React, Node.js)
- Status: runs locally on Windows; deployment kit prepared (Docker, Caddy), not deployed
- Data note: estimator rates are SAMPLE values (rates.is_sample = 1)

Table 1. Project facts

Item	Value
Repository layout	src/ (Next.js), services/ (6 services + db), docs/, deploy/
Ports	3001, 5173, 8000, 8001, 8080, 8081, 4000; MySQL 3306; MongoDB 27017
Code size	not measured; do not quote a line count
Reference estimate	MS 2 mm, 500 × 300 mm, cut 1600 mm, 4 pierces, 2 bends, powder coat, qty 10 → INR 2,933.83

Explain (technical notes for the speaker)

- Seven applications: Next.js Web Application, React Portal Application, PHP Estimator Application, Laravel Admin Application, Servlet/JSP Desk Application, Spring Boot Core API, Node.js Enquiry API.
- Two data stores: MySQL database (pslaser) as the system of record, MongoDB activity log for audit events.
- Be explicit that machines, services and contact data are real, but price rates are placeholders until the owner supplies the rate card.

Part 2 — Introduction

Slide 3 — Introduction

On the slide

- A laser-cutting job shop prices work from material, thickness, cut length, pierces, bends and finishing
- PS Laser Web Platform automates this as a set of cooperating services over one shared schema
- Contract-first design: one written contract fixes schema, JWT, price formula and API shapes
- Heterogeneous stack by design: each service demonstrates one syllabus technology on real business data

Explain (technical notes for the speaker)

- Technical framing: the platform is a request/response system. Clients send estimate, enquiry, tracking and login requests; staff users send updates; the system persists state in MySQL and emits events.
- Why seven services: the course units cover PHP, Laravel, React, Servlet/JSP, Spring Boot, Advanced React and Node.js. Instead of seven toy demos, each technology implements a real feature of one business domain.
- The risk of such a design is inconsistency. It is controlled by services/CONTRACT.md and by one reference test case that every implementation must reproduce.

Slide 4 — Business and machine context

On the slide

- Equipment: HSG 3 kW and Genesis 1.5 kW fiber lasers (3 m × 1.5 m beds), 175 t CNC press brake, Amada 32 t and 16 t turret punches, 1.5 kW handheld laser welder, PLC powder-coating plant
- Laser capacity: mild steel 16 mm, stainless steel 12 mm, aluminium 8 mm, brass 4 mm, copper 4 mm
- The software has no interface to the machines; job status is entered by staff users

Explain (technical notes for the speaker)

- This slide explains where the cost-model parameters come from: bed size 3000 × 1500 mm is the validation limit, and the maximum thickness per material is stored in the materials table.
- Source: company profile PDF, summarised in docs/research/pslaser/brief.md. Product and gallery pictures are representative renders, not photographs of finished jobs.

Part 3 — Problem Statement, Aim and Objectives

Slide 5 — Problem statement

On the slide

- P1 No automated computation of an indicative price from part parameters
- P2 Enquiry intake is unstructured: requirements and drawing files arrive over telephone and chat
- P3 Job progress is not exposed as queryable state
- P4 Prices and job status are not maintained through a controlled, role-based interface
- P5 Several heterogeneous technology stacks must stay consistent on one data model

Explain (technical notes for the speaker)

- P1: price depends on area, weight, cut length, pierces, bends and coating. A manual calculation is slow and not reproducible.
- P2: the shop's published channels are telephone and WhatsApp (four phone numbers and one WhatsApp number on the contact page). Files and requirements are not stored as structured records.
- P3: no data model for job stages existed; the platform introduces work_orders and work_order_events with seven stages.
- P5 is the course-level problem: PHP, Java, TypeScript and Node implementations must return identical money values.
- Avoid claiming industry-wide statistics. State that the problem statement is specific to this shop.

Slide 6 — Aim

On the slide

- To design and implement a contract-driven, multi-service web platform that computes indicative estimates, captures enquiries with drawing files, tracks work orders through seven stages and gives staff users role-based control, using the technologies of CS3111 Units I–VI.

Explain (technical notes for the speaker)

- Read the aim as one sentence. Each clause maps to a module: estimates (M2), enquiries and files (M3), work-order tracking (M5, M6, M7), role-based control (M4, M8).

Slide 7 — Objectives

On the slide

- O1 Implement one estimate formula identically in PHP, Java (Spring and Servlet) and TypeScript
- O2 Expose REST APIs with JWT authentication (Spring Boot) and validated file ingestion (Node.js)
- O3 Model work orders with seven stages and an append-only status-event history
- O4 Push status changes to a dashboard in real time (STOMP) with a polling fallback
- O5 Build a role-based administration application (Laravel) for rates, catalogue and enquiries

- O6 Apply Advanced React: TypeScript, SSR/SSG/ISR, hooks, state library, performance tuning
- O7 Keep all services consistent through one schema, one JWT secret and one contract
- O8 Verify correctness with cross-stack equivalence tests and automated suites

Explain (technical notes for the speaker)

- Each objective is testable: O1 by the reference case, O2 by 401/403 and upload tests, O3 by status-event rows, O4 by the STOMP topic, O8 by the test results on Slide 49.

Part 4 — Literature Review

Slide 8 — Literature review: foundations used

On the slide

- REST architectural style, MVC, JWT, bcrypt, WebSocket, OWASP risks, ISO/IEC 25010 quality model, iterative-incremental development
- Table lists each source and how the platform applies it

Table 2. Literature and application in PS Laser

Source	Contribution	Applied in the platform
R. T. Fielding, 2000 (REST dissertation)	Defines REST constraints: stateless client-server, uniform interface	Spring and Node.js APIs, resource URLs, HTTP verbs and status codes
Krasner and Pope, 1988 (MVC)	Separation of model, view and controller	Laravel, Servlet/JSP and Spring layering (controller → service → repository)
RFC 7519 JWT; RFC 7518 JWA (HS256)	Signed, self-contained tokens; HMAC-SHA-256	Stateless authentication shared by Spring and Node.js
Provos and Mazières, 1999 (bcrypt)	Adaptive password hashing	users.password stored as bcrypt $2y$ hashes
RFC 6455 WebSocket	Full-duplex client-server channel	STOMP over WebSocket for the live dashboard
OWASP Top 10:2021	Catalogue of web-application risks	Access control, input validation, upload checks, CSRF tokens, rate limiting
ISO/IEC 25010:2011	Software quality characteristics	Non-functional requirements in the requirements document
Larman and Basili, 2003	History of iterative and incremental development	Development model (Slide 46)
DeMarco, 1978; Gane and Sarson, 1979	Data-flow diagram notation	DFD Level 0, 1 and 2 (Slides 17–21)

Explain (technical notes for the speaker)

- Be honest about scope: these are foundational and standards sources. Verifiable domain-specific papers on small-shop quotation portals were not identified, so no novelty claim is made about the estimator itself.
- Before presenting, search Google Scholar for "sheet metal cost estimation", "web-based quotation system manufacturing" and "instant quoting laser cutting". Add only papers you have opened and read.

Slide 9 — Comparison of architectural approaches and gap

On the slide

- Monolith vs microservices vs multi-stack shared-schema (this project)
- Shared database trades coupling for simplicity; mitigated by a contract and one reference test
- Gap addressed: a single business domain implemented consistently across PHP, Java, TypeScript and Node.js

Table 3. Architectural approaches

Approach	Strength	Weakness	Fit here
Monolith	One deployment, simple	Cannot show 6 different stacks	Rejected: syllabus needs separate technologies
Microservices, DB per service	Loose coupling	Data duplication, heavy operations	Rejected: too heavy for a course project
Multi-stack, shared schema + contract	Each unit stands alone; one data model	Duplicated rules; schema changes touch all services	Chosen, with contract and reference test

Explain (technical notes for the speaker)

- Microservice literature (for example Newman, Building Microservices, and Richardson, Microservices Patterns) normally recommends a database per service. This project deliberately shares one MySQL schema, because the course requires each unit to be independently demonstrable and a single data model keeps the demonstration small.
- The cost of sharing is duplicated business rules in four estimators. The platform controls it with one formula specification, one rounding rule and one reference case that returns INR 2,933.83 on Spring, Next.js, PHP and the Servlet application [V, 14 Oct and 16 Oct 2026].
- State the gap carefully: it is an engineering-integration gap for a course project, not a research gap in cost estimation.

Part 5 — Technologies

Slide 10 — Technology stack overview

On the slide

- Presentation: Next.js 16 (React 19, TypeScript), React 19 + Vite + Redux Toolkit, PHP 8.2, Laravel 12, Servlet/JSP
- Application: Spring Boot 3.5 (Java 21), Node.js 24 with Express 5
- Data: MySQL/MariaDB 10.4, MongoDB
- Security: JWT HS256, bcrypt, CSRF tokens, CORS allow-list, zod validation

Table 4. Stack by layer

Layer	Technologies (versions)
Presentation	Next.js 16.3.5, React 19.2, TypeScript 5 (strict), Tailwind CSS 4, Bootstrap 4.5, Zustand 5; React Portal: React 19.2, Vite 8, Redux Toolkit 2, React Router 7; PHP 8.2; Laravel 12 views; JSP with JSTL
Application	Spring Boot 3.5.16 (web, validation, data-jpa, security, websocket, actuator), jjwt 0.12.6; Node.js 24 with Express 5.3, zod 4, multer 2.4, helmet 8, express-rate-limit 8, jsonwebtoken 9, mysql2 3; Laravel 12 on PHP 8.2; Tomcat 8.5 with JDBC
Data	MySQL/MariaDB 10.4.32 (database pslaser, 11 shared tables + framework tables); MongoDB (embedded or Docker) for activity_log; private file storage on disk
Tooling	npm 11, Maven wrapper, Composer 2.10, ESLint, TypeScript compiler, JUnit 5 + Mockito + MockMvc, node:test + supertest, PHPUnit, Postman, Git

Explain (technical notes for the speaker)

- Walk the stack bottom-up: data, application services, presentation applications. Mention that every version number on the slide is taken from package.json, pom.xml, composer.json or the XAMPP installation on the development machine.

Slide 11 — Frontend technologies and why

On the slide

- Next.js 16: App Router, server components, route handlers, server actions, SSG/ISR/SSR
- TypeScript strict: typed DTOs, discriminated unions, runtime validation
- Zustand 5 for the quote cart; Redux Toolkit in the React Portal Application
- Tailwind 4 and Bootstrap 4 for the classic responsive layout; PHP and JSP views render server-side

Explain (technical notes for the speaker)

- Next.js was chosen for Unit V because it gives server-side rendering, streaming with Suspense and a BFF layer in one framework.
- Two state libraries are used on purpose: Redux Toolkit (Unit II) and Zustand (Unit V). Zustand fits small client state with few actions (selector subscriptions, about 1 kB); Redux Toolkit suits larger action-driven state.
- The React Portal Application is Vite + React and talks directly to the Core API and the Enquiry API from the browser.

Slide 12 — Backend technologies and why

On the slide

- Spring Boot Core API: layered controller → service → repository, JPA, Spring Security, STOMP, Actuator
- Node.js Enquiry API: Express 5, zod validation, multer uploads, helmet, rate limiting, file-system logging
- PHP Estimator Application: plain PHP 8.2 with PDO, sessions, cookies, CSRF token
- Laravel Admin Application: MVC, Eloquent, Artisan, role middleware, audit table
- Servlet/JSP Desk Application: Tomcat 8.5, sessions, URL rewriting, filter, JDBC

Explain (technical notes for the speaker)

- Spring is the authoritative implementation of the estimate formula and the only issuer of JWTs. Node.js verifies the same JWT with the shared secret.
- Money values use BigDecimal in Java with half-up rounding. The TypeScript copy uses a round2 helper that follows the same rule.

Slide 13 — Data, infrastructure and tooling

On the slide

- MySQL/MariaDB: single system of record, InnoDB, foreign keys, ENUM status columns
- MongoDB: schema-less activity_log written by Spring event listeners
- Private disk storage for drawing files (random file names, not served by URL)
- Docker, Caddy and docker-compose: deployment kit prepared, not deployed

Explain (technical notes for the speaker)

- MongoDB data is lost on restart when the embedded instance is used (profile mongo-embedded); the production kit uses a persistent MongoDB container.
- Docker Desktop was not running on the development PC, so the container images were never built. Say this plainly if asked.

Slide 14 — Technology to syllabus unit map

On the slide

- Unit I: PHP Estimator Application, Laravel Admin Application
- Unit II: React Portal Application (components, Router, Redux Toolkit)
- Unit III: Servlet/JSP Desk Application (servlets, JSP, sessions, cookies, URL rewriting, JDBC)
- Unit IV: Spring Boot Core API (JWT, JPA, SQL + MongoDB, STOMP, Actuator, Postman)
- Unit V: Next.js Web Application (TypeScript, SSR, hooks, state library, patterns)
- Unit VI: Node.js Enquiry API (modules, NPM, Express, file system, static files, database)

Explain (technical notes for the speaker)

- Point to docs/SYLLABUS-MAPPING.md and docs/syllabus/unit-*.md: they list every syllabus topic with file and line references for the viva.

Part 6 — System Architecture (with DFD)

Slide 15 — System architecture overview

On the slide

- Five tiers: client, presentation (5 applications), API (2 services), data (MySQL, MongoDB, file storage), physical shop (context only)
- Presentation applications reach data either through APIs or directly over SQL
- Spring publishes domain events; Node.js stores files on private disk

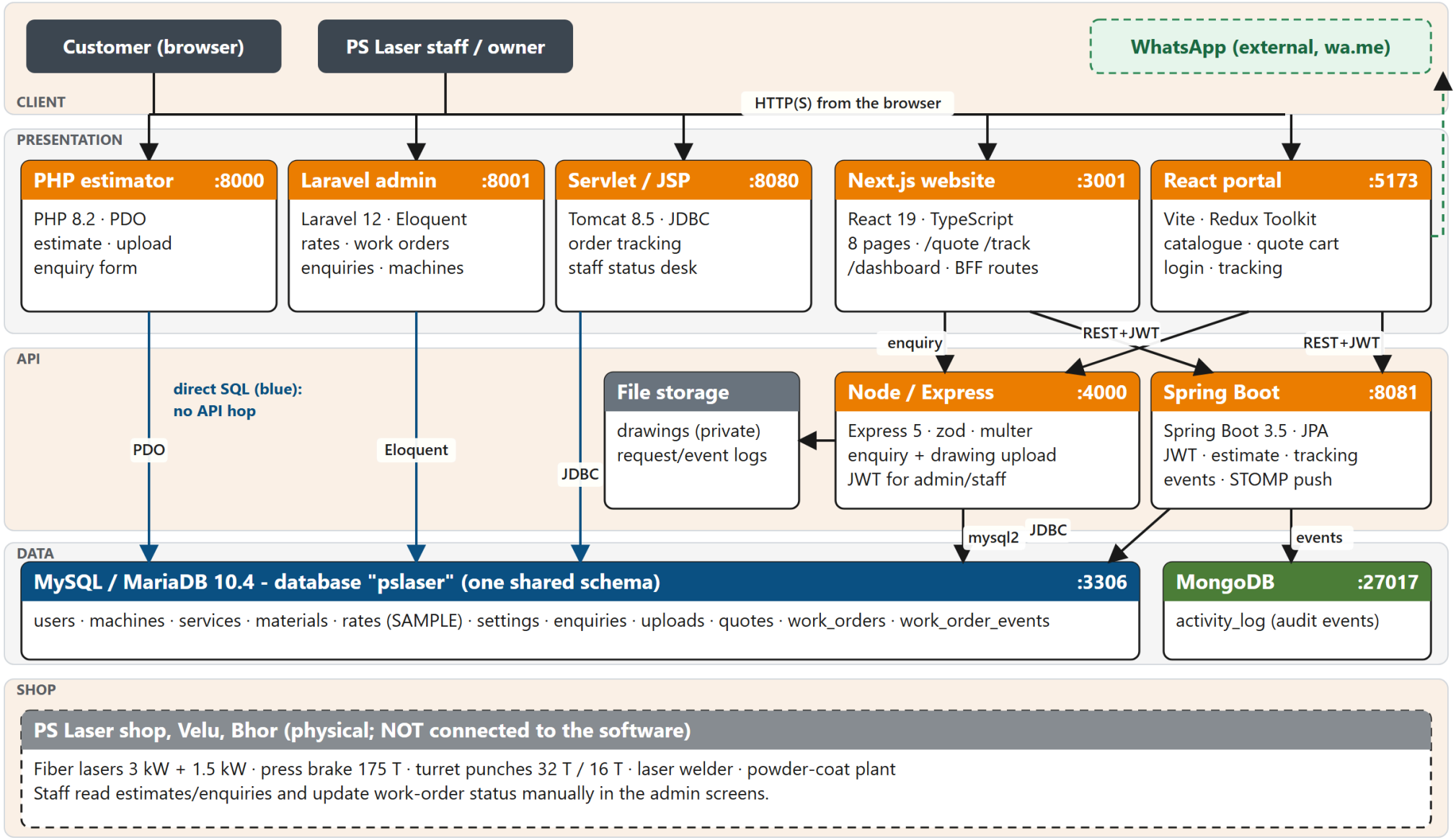


Figure 1. System architecture (file: docs/submission/PSLaser_Architecture_Diagram_detailed.png has the legend version)

Explain (technical notes for the speaker)

- Direct SQL access (PHP, Laravel, Servlet/JSP) keeps each syllabus unit self-contained. API access (Next.js, React Portal) uses REST with JWT.
- The physical shop is dashed on the diagram: no software interface to the machines exists.
- The browser reaches only the presentation applications and the Spring WebSocket endpoint; MySQL, MongoDB and the Node.js API are not exposed in the production kit.

Visual / file: docs/submission/PSLaser_Architecture_Diagram_detailed.png (full detail) or fig-architecture-compact.png.

Slide 16 — Architectural decisions

On the slide

- Contract-first: services/CONTRACT.md before code
- Shared schema with direct SQL for three applications
- Backend-for-Frontend route handlers in Next.js
- Graceful degradation: offline TypeScript estimate, demo data in the portal
- Stateless authentication with HS256 JWT (8 h)
- Event-driven side effects through domain events
- One rounding rule: round each line half-up to 2 decimals, then add

Explain (technical notes for the speaker)

- Each decision answers a risk: contract-first reduces integration drift; BFF keeps the Spring URL and JWT out of the browser bundle; domain events decouple logging and push from business logic; the rounding rule removes one-paisa differences between languages.
- Events are published inside the transaction and delivered after commit, so a rolled-back status change never produces a push.

Slide 17 — DFD Level 0 — context diagram

On the slide

- One process: PS Laser Web Platform
- External entities: E1 Client, E2 Staff user, E3 WhatsApp service
- Inputs: requests (page, estimate, enquiry + file, orderNo + phone, credentials); staff updates (JWT)
- Outputs: HTML/JSON, cost breakdown, id + wa.me URL, status timeline, JWT, live push

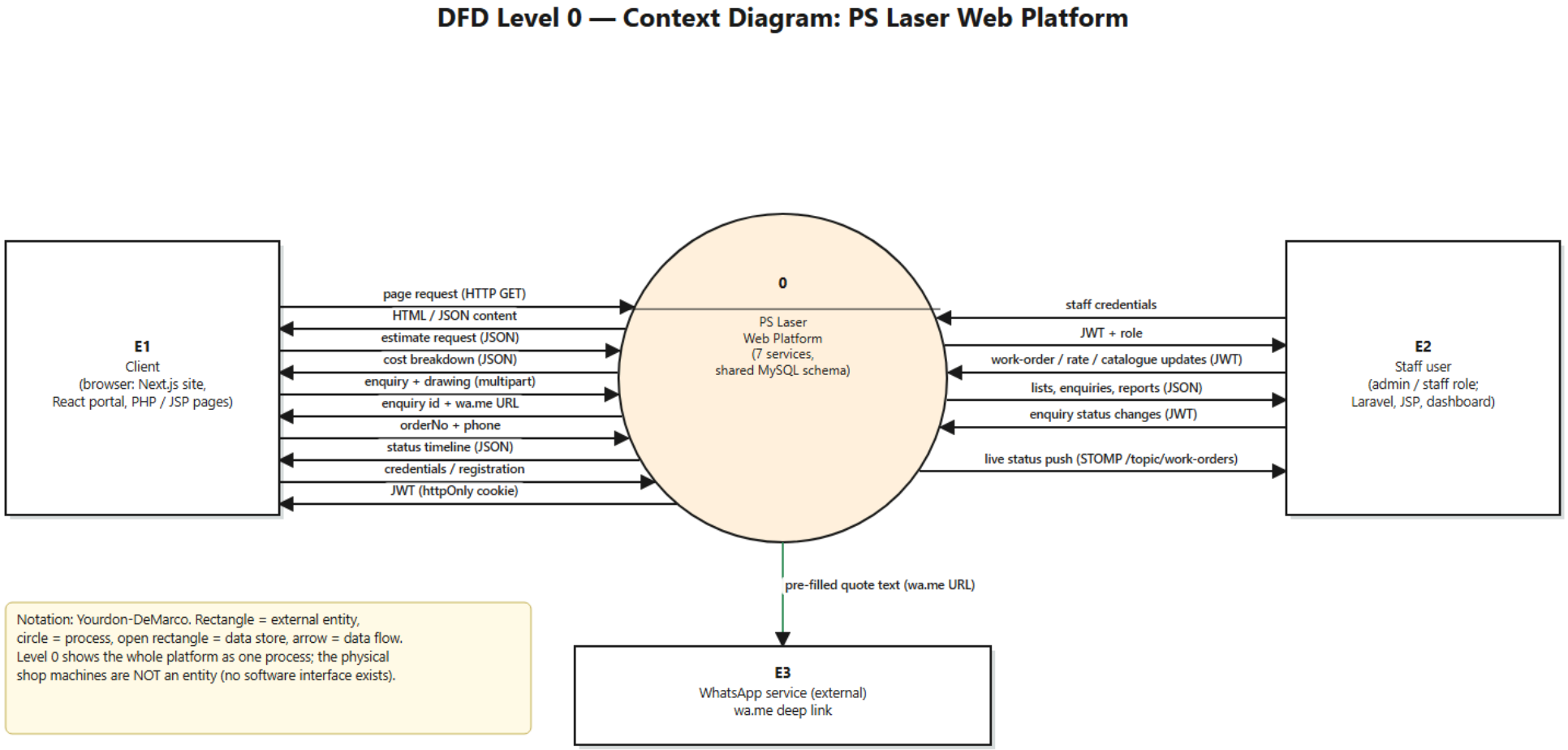


Figure 2. DFD Level 0

Explain (technical notes for the speaker)

- Explain the notation: rectangle = external entity, circle = process, open rectangle = data store, arrow = data flow.
- WhatsApp is an external service reached through a URL (wa.me deep link); there is no server-to-server integration.

Visual / file: docs/ppt/_build/dfd-level0.png

Slide 18 — DFD Level 1 — decomposition

On the slide

- 7 processes: 1.0 Serve Content and Catalogue, 2.0 Compute Estimate, 3.0 Capture Enquiry and Drawing, 4.0 Authenticate and Authorise, 5.0 Track Order, 6.0 Manage Work Orders, Rates, Catalogue, 7.0 Log and Broadcast Events
- 8 data stores: D1 Users, D2 Catalogue, D3 Rate card, D4 Enquiries, D5 Quotes, D6 Work orders, D7 Drawing files, D8 Activity log
- Red dashed arrows are domain events into 7.0

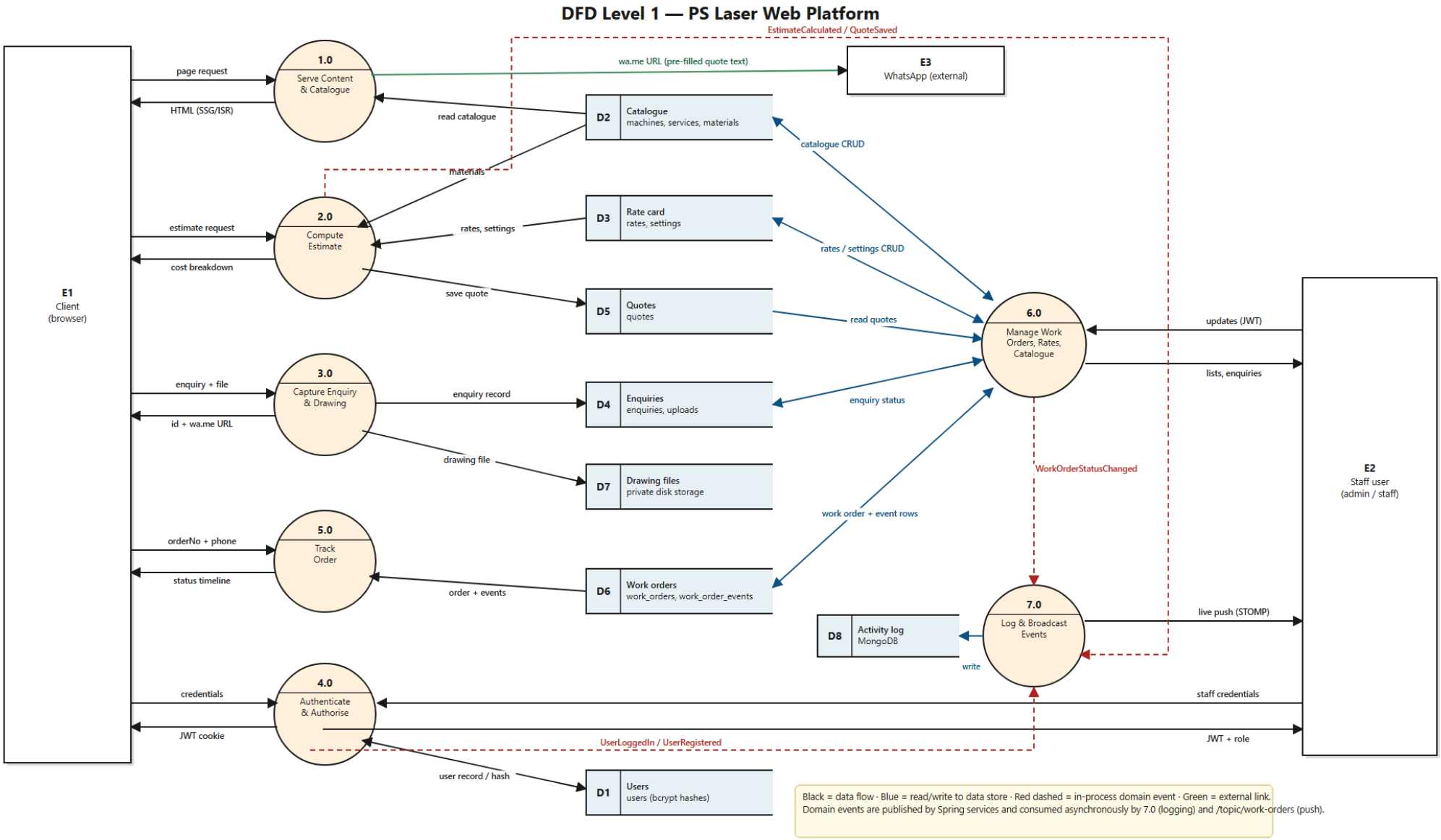


Figure 3. DFD Level 1

Explain (technical notes for the speaker)

- Walk one path end to end: Client → 2.0 → reads D2 and D3 → returns cost breakdown → emits EstimateCalculated → 7.0 writes D8.
- Then a second path: Staff user → 4.0 (JWT) → 6.0 → writes D6 → WorkOrderStatusChanged → 7.0 → pushes to Staff user and writes D8.

Visual / file: docs/ppt/_build/dfd-level1.png (1960 px wide; insert full slide).

Slide 19 — DFD Level 1 — process and data-store dictionary

On the slide

- Table lists each process with inputs, outputs, data stores and implementing service

Table 5. DFD Level 1 dictionary

Element	Description	Inputs → Outputs	Stores	Implemented by
1.0	Serve content and catalogue	page request → HTML; wa.me URL	D2 (r)	Next.js Web Application; Spring/Node catalogue endpoints
2.0	Compute estimate	EstimateInput → EstimateResult	D2, D3 (r); D5 (w)	Spring EstimateService; TypeScript, PHP, Servlet copies
3.0	Capture enquiry and drawing	JSON or multipart → id, wa.me URL	D4, D7 (w)	Node.js Enquiry API; PHP Estimator Application
4.0	Authenticate and authorise	credentials → JWT/session + role	D1 (r/w)	Spring Security; Laravel and Servlet sessions
5.0	Track order	orderNo + phone → status timeline	D6 (r)	Spring tracking endpoint; Next.js /track; JSP /track
6.0	Manage work orders, rates, catalogue	staff updates → persisted rows; lists	D2, D3, D4, D5, D6	Laravel Admin; Spring admin endpoints; Servlet/JSP desk
7.0	Log and broadcast events	domain events → activity record; STOMP push	D8 (w)	Spring ActivityEventListener
D1–D8	Data stores	see Slide 22	—	MySQL database (D1–D6), disk (D7), MongoDB (D8)

Explain (technical notes for the speaker)

- Use this table as the speaker script for Slide 18. Each row is one DFD element.

Slide 20 — DFD Level 2 — process 2.0 Compute Estimate

On the slide

- 2.1 Validate Request → 2.2 Resolve Material and Rate Band → 2.3 Compute Cost Lines → 2.4 Round, Quantity and GST → 2.5 Compose Response; 2.6 Persist Quote (optional)
- Error flows: HTTP 400 with fieldErrors; 400 no rate band
- Reads D2 (materials) and D3 (rates, settings); emits EstimateCalculated

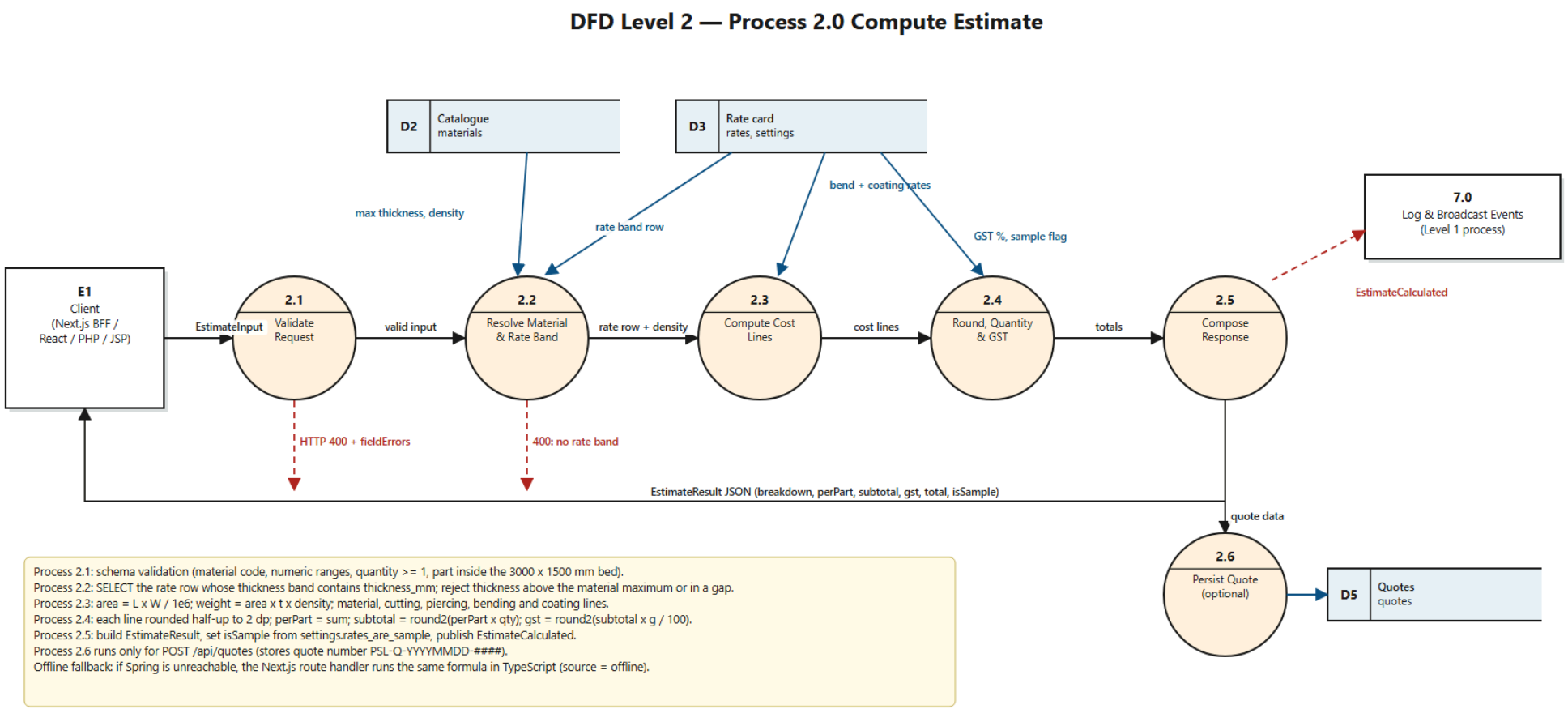


Figure 4. DFD Level 2: Compute Estimate

Explain (technical notes for the speaker)

- Validation covers material code, numeric ranges, quantity at least 1 and the 3000 × 1500 mm bed. A thickness above the material maximum or inside a gap between rate bands is rejected.
- The same chain exists in four languages; the chain order is the same in each, which is why one reference case can test all.

Visual / file: docs/ppt/_build/dfd-level2-estimate.png

Slide 21 — DFD Level 2 — work-order lifecycle (6.0, 7.0, 5.0)

On the slide

- 6.1 Verify JWT and Role → 6.2 Create/Update Work Order → 6.3 Append Status Event (transaction) → 6.4 Publish Status Changed Event
- 7.1 Persist Activity Record (MongoDB); 7.2 Broadcast STOMP message
- 5.1 Validate orderNo + phone → 5.2 Fetch Order + Events → 5.3 Mask Phone, Build Timeline

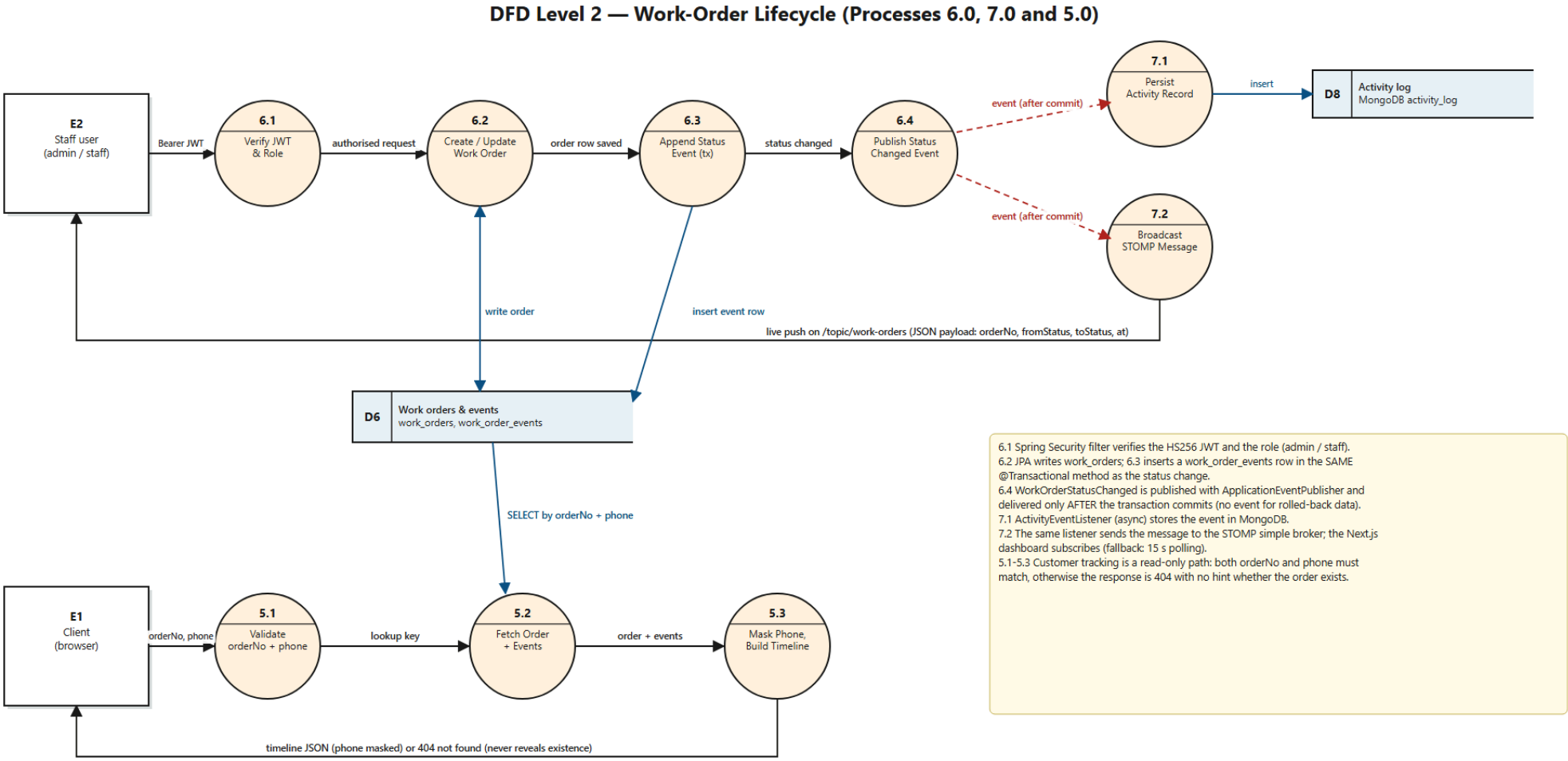


Figure 5. DFD Level 2: work-order lifecycle

Explain (technical notes for the speaker)

- 6.3 inserts the status-event row in the same @Transactional method as the status change; 6.4 publishes WorkOrderStatusChanged and it is delivered after commit.
- The tracking path is read-only. Both orderNo and phone must match; otherwise the response is 404 and does not reveal whether the order exists. The phone number is returned masked.

Visual / file: docs/ppt/_build/dfd-level2-orders.png

Slide 22 — Data model (MySQL database pslaser)

On the slide

- 11 shared tables: users, machines, services, materials, rates, settings, enquiries, uploads, quotes, work_orders, work_order_events
- Foreign keys: rates → materials; quotes → materials, enquiries; uploads → enquiries; work_order_events → work_orders (ON DELETE CASCADE)
- ENUM columns: users.role, enquiries.status and source, work_orders.status
- Laravel adds framework tables (sessions, cache, jobs, admin_audit_logs) without touching the shared tables

Table 6. Key tables and columns

Table	Key columns
users	id, name, email (unique), phone, password (bcrypt), role ENUM(admin, staff, customer)
materials	code (unique), name, max_thickness_mm, density_g_cm3
rates	material_code → materials.code, thickness_from_mm, thickness_to_mm, cut_rate_per_m, pierce_rate, material_rate_per_kg, is_sample
settings	key (PK), value: bend_rate_inr, powder_coat_rate_per_sqm, gst_percent, rates_are_sample
enquiries	id, name, phone, email, service_slug, machine_slug, message, source ENUM, status ENUM(new, contacted, quoted, won, lost)
uploads	id, enquiry_id → enquiries (SET NULL), original_name, stored_path, mime, size_bytes
quotes	quote_no (unique, PSL-Q-YYYYMMDD-####), material_code, thickness_mm, length_mm, width_mm, cut_length_mm, pierces, bends, powder_coat, quantity, amount_inr
work_orders	order_no (unique), customer_name, customer_phone, description, machine_slug, quantity, status ENUM(7 stages), due_date
work_order_events	work_order_id → work_orders (CASCADE), status, note, created_at

Explain (technical notes for the speaker)

- Why work_orders and not jobs: Laravel owns a table called jobs; the name was avoided to prevent a clash.
- Estimator rates are stored per material and thickness band (thickness_from_mm to thickness_to_mm). settings holds bend_rate_inr, powder_coat_rate_per_sqm, gst_percent and rates_are_sample.
- Hibernate runs with ddl-auto = validate so JPA entities are checked against the schema but never alter it.

Slide 23 — Unit V — front-end layered architecture (Next.js Web Application)

On the slide

- L1 Presentation (pages, feature components) → L2 Logic (custom hooks, reducer, Zustand) → L3 Service (lib/api) → L4 BFF route handlers and backends
- Feature-based folders: features/quote, features/track, features/dashboard
- Browser never talks to MySQL; server-only modules (spring.ts, session.ts) never reach the client bundle

Front-End Layered Architecture — Next.js Web Application (Unit V mapping)

L1 Presentation Layer

pages + components

app/(classic)/quote/page.tsx (SSG + ISR shell), track/page.tsx (SSR + Suspense), dashboard/page.tsx (SSR, HOC)
features/quote/* · features/track/* · features/dashboard/* (feature folders, barrel index.ts)
components/advanced/*: ErrorBoundary (class), PortalModal, VirtualList<T>, AsyncView (render props), Panel (polymorphic)
Patterns: compound components (Estimator.Number), container/presenter, controlled/uncontrolled inputs

L2 Logic Layer

custom hooks + client state

hooks/use-estimate.ts (AbortController, race guard) · use-debounced-value.ts (350 ms) · use-query.ts (cache hook)
hooks/use-live-work-orders.ts (STOMP + 15 s polling) · use-local-storage.ts · use-media-query.ts
features/quote/quote-reducer.ts (useReducer state machine) · store/quote-store.ts (Zustand + persist)
Performance: React.memo, useMemo, useCallback, useTransition, useDeferredValue, lazy + Suspense, next/dynamic

L3 Service Layer

lib/api (isomorphic + server-only)

lib/api/client.ts: apiFetch<T>(url, schema), withFallback<T> · lib/api/validate.ts + schemas.ts (runtime validation)
lib/api/query-cache.ts (stale-while-revalidate, retry 500 ms → 1 s → 2 s) · lib/api/stomp.ts (STOMP 1.2 client)
lib/api/estimate.ts (TypeScript copy of the formula = offline fallback)
server-only: lib/api/spring.ts (Bearer JWT, no-store / revalidate tags), lib/api/session.ts (verifyJwt HS256)

L4 Backend-for-Frontend + Backends

server side

BFF route handlers: app/api/estimate, app/api/auth/login|logout, app/api/work-orders (validate, proxy, httpOnly cookie)
Server Action: features/quote/actions.ts "use server" → Node.js Enquiry API (URL never reaches the browser bundle)
Spring Boot Core API :8081 (JWT, JPA, STOMP) · Node.js Enquiry API :4000 (zod, multer) · MySQL pslaser · MongoDB
Rule: the browser never talks to MySQL; the Web Application talks to APIs through L3 only

Feature-based folder layout (src/)

```
app/ routes (App Router), route handlers
(classic)/ main site: quote, track, dashboard
api/ BFF handlers
features/ one folder per business feature
quote/ track/ dashboard/
components/advanced/ shared reusable components
hooks/ shared custom hooks
store/ Zustand store
lib/api/ service layer + types
lib/pslaser/data.ts content constants
```

Rendering strategy per route

/quote
SSG + ISR (revalidate 3600 s) + CSR islands

/track
SSR per request, streamed (Suspense)

/dashboard
SSR (cookie) + CSR live board

/api/*
dynamic route handlers (BFF)

other pages
static (prerendered at build)

Verified by production build (21 Oct 2026):
Next.js 16.3.5 (Turbopack): compiled in 12.3 s, TypeScript check 10.2 s, 22 static pages in 1.5 s; /quote marked Revalidate 1h; /track, /dashboard, and /api/* marked ƒ (dynamic).

Figure 6. Front-end layered architecture

Explain (technical notes for the speaker)

- This mirrors the layered architecture slide of the Unit V deck (Presentation, Logic, Service, Backend) and its feature-based architecture slide (features/ with components, hooks, services).
- Barrel files (index.ts) expose each feature; isomorphic modules are exported from lib/api/index.ts, while server-only modules are imported by path so the bundler cannot include them in client code.

Visual / file: docs/ppt/_build/unit5-layers.png

Slide 24 — Unit V — advanced component patterns used

On the slide

- Compound components, render props, higher-order component, container/presenter, controlled/uncontrolled, portals, error boundary, polymorphic prop, custom hooks
- Each pattern maps to a file in src/ (table)

Table 7. Unit V pattern to implementation

Unit V topic	Implementation in src/
Compound components	features/quote/estimator.tsx: Estimator + .Number, .Material, .Toggle, .Row sharing a typed context
Render props	components/advanced/async-view.tsx (success render prop); virtual-list.tsx (renderRow); error-boundary.tsx (fallback(error, reset))
Higher-order component	features/dashboard/with-auth.tsx: withAuth(Component, { roles, fallback }); used in app/(classic)/dashboard/page.tsx
Container / presenter	container: features/quote/quote-calculator.tsx; presenters: estimate-summary.tsx, features/track/order-timeline.tsx
Controlled / uncontrolled	hooks/use-controllable-state.ts + quantity-stepper.tsx (both modes); enquiry-panel.tsx uncontrolled (FormData); track-form.tsx uncontrolled GET form
Custom hooks	use-estimate, use-debounced-value, use-query, use-live-work-orders, use-local-storage, use-media-query, use-controllable-state
Component composition	components/advanced/panel.tsx (polymorphic "as" prop), portal-modal.tsx (children)
Error boundaries	components/advanced/error-boundary.tsx (class); app/(classic)/track/error.tsx (route-level)
Portals	components/advanced/portal-modal.tsx: createPortal(…, document.body)
Layered / feature architecture	features/*, hooks/, lib/api/ (service layer), store/, components/advanced/

Explain (technical notes for the speaker)

- The Unit V deck lists: HOC, render props, compound components, controlled and uncontrolled components, custom hooks, composition, error boundaries and container/presentational components. Every one of them exists in the code base.
- The HOC is a server-side higher-order component: withAuth(Page) verifies the httpOnly JWT cookie on the server and injects the session.

Slide 25 — Unit V — rendering strategies and performance tuning

On the slide

- SSG + ISR: /quote (revalidate = 3600); SSR + Suspense streaming: /track; SSR + client island: /dashboard; BFF route handlers: /api/*
- React.memo, useMemo, useCallback, useTransition, useDeferredValue, React.lazy + Suspense, next/dynamic, own list virtualisation, Profiler
- State: useReducer, Context, Zustand with persist and selectors; own SWR-style query cache

Table 8. Route to rendering strategy [V build output, 21 Oct 2026]

Route	Strategy	Build marker
/, /about-us, /gallery, /blogs …	Static prerender at build	○ Static
/blogs/[slug] (3 articles)	Static HTML with generateStaticParams	● SSG
/quote	SSG shell + ISR (3600 s) + client islands	Revalidate 1h
/track	SSR per request, Suspense streaming, error.tsx	*ƒ* Dynamic
/dashboard	SSR (httpOnly cookie) + CSR live board	*ƒ* Dynamic
/api/estimate, /api/auth/*, /api/work-orders	Route handlers (BFF), never cached	*ƒ* Dynamic

Explain (technical notes for the speaker)

- SSR renders HTML per request (fresh, per-user data: /track, /dashboard). SSG renders once at build. ISR is SSG with background regeneration after revalidate seconds. CSR runs in the browser after hydration (calculator, live board).
- Virtualisation: quantity 5000 produces a parts list of 5000 rows but only about 20 DOM rows are rendered. This was observed during development (docs/syllabus/unit-5-advanced-react.md).
- Verified production build on 21 Oct 2026: Next.js 16.3.5 Turbopack compiled in 12.3 s; 22 static pages generated in 1.5 s; /quote shows Revalidate 1h; /track, /dashboard and /api/* are dynamic [V].

Slide 26 — Security architecture

On the slide

- Authentication: HS256 JWT (8 h) issued by Spring; verified by Spring and Node.js with a shared secret
- Authorisation: roles admin, staff, customer; /api/admin/** requires admin or staff; Laravel allows delete only for admin
- Credentials: bcrypt $2y$ hashes; httpOnly + SameSite=Lax cookie in the Next.js Web Application
- Input: zod schemas and prepared statements (Node.js); bean validation (Spring); CSRF tokens (PHP, Laravel)
- Uploads: type allow-list, 10 MB limit, content check, private storage; rate limit 20 enquiries per 15 minutes by default
- Tracking requires orderNo + phone; phone returned masked; wrong phone → 404

Explain (technical notes for the speaker)

- Spring uses SessionCreationPolicy.STATELESS with a JwtAuthFilter. Public routes are listed explicitly (login, register, catalogue, estimate, tracking); everything else under /api/admin requires ADMIN or STAFF.
- Known limit: development defaults exist (root database user with empty password, a development JWT secret). The deployment kit refuses to start without real secrets.

Slide 27 — Deployment view

On the slide

- Local topology (verified): 8 server processes on one Windows PC, ports 3001, 4000, 5173, 8000, 8001, 8080, 8081, 3306
- Prepared production topology: Caddy (HTTPS) → Next.js Web Application, Laravel Admin Application, Spring WebSocket path; MySQL, MongoDB, Spring REST and Node.js stay on a private network
- Status: prepared in deploy/, not built or deployed

Explain (technical notes for the speaker)

- Be precise: docker compose config passes and the shell scripts pass a syntax check, but no image was built because Docker Desktop was not running. Domain pslaser.online is available; the DNS records and a server are still needed.

Part 7 — Methodology (all modules: description and workflow)

Slide 28 — Methodology overview and module list

On the slide

- Approach: iterative-incremental, contract-first; one module per increment
- Build order: contract and schema → Spring Boot Core API → Node.js Enquiry API → Next.js Web Application → PHP, Laravel, Servlet/JSP, React Portal
- 11 modules (table)

Table 9. Module list

ID	Module	Owner service(s)	Unit	Stores
M1	Content and Catalogue Delivery	Next.js Web Application; Spring/Node catalogue	V	D2
M2	Estimation Engine	Spring Core API (authoritative); TypeScript, PHP, Servlet copies	IV, V, I, III	D2, D3, D5
M3	Enquiry and Drawing Ingestion	Node.js Enquiry API; PHP Estimator Application	VI, I	D4, D7
M4	Authentication and Access Control	Spring Security; Laravel; Servlet filter; Next.js cookie	IV, I, III, V	D1
M5	Order Tracking	Spring; Next.js /track; Servlet/JSP	IV, V, III	D6
M6	Work-Order Management	Spring admin; Laravel; Servlet/JSP desk	IV, I, III	D6
M7	Event Processing and Live Updates	Spring listener, STOMP, MongoDB; Next.js dashboard	IV, V	D8
M8	Administration and Rate-Card Management	Laravel Admin Application	I	D2, D3, D4, D5, D6
M9	Customer Portal	React Portal Application (Redux Toolkit)	II	via APIs
M10	Stand-alone Server-Side Applications	PHP Estimator; Servlet/JSP Desk	I, III	D4, D5, D6
M11	Shared Contract and Data Layer	CONTRACT.md, schema.sql, seed.sql	all	all

Explain (technical notes for the speaker)

- Explain that every module has an owner service, a set of data stores and a workflow of numbered steps. The next slides give description and workflow for each module.
- Start order at runtime is MySQL → Spring → Node.js → Next.js → the rest.

Slide 29 — M1 — Content and Catalogue Delivery Module

On the slide

- Description: serves 8 public pages and catalogue data (7 machines, 8 services, 5 materials)
- Rendering: static prerender at build; /quote ISR 3600 s
- Quote action: wa.me deep link with a context-specific pre-filled message

Explain (technical notes for the speaker)

- Workflow:
- 1. Client sends HTTP GET for a page.
- 2. The Next.js Web Application returns prerendered HTML from the build output (SSG); interactive islands hydrate.
- 3. For /quote, a server component fetches materials from the Spring Core API with next.revalidate = 3600 and the cache tag "materials"; on failure it uses the constant data in lib/pslaser/data.ts.
- 4. Catalogue data also exists as REST resources: GET /api/machines, /api/services, /api/materials (Spring) and GET /api/catalogue (Node.js, cached 60 s by default).
- 5. A quote button builds https://wa.me/917719976990?text=… with encodeURIComponent and opens it. Nothing is stored for this path.
- Inputs: HTTP GET. Outputs: HTML, JSON. Store read: D2.

Slide 30 — M2 — Estimation Engine Module

On the slide

- Description: computes an indicative estimate from material, thickness, size, cut length, pierces, bends, powder coat flag and quantity
- One formula, four implementations (Spring authoritative; TypeScript offline fallback; PHP; Servlet)
- Validation: material exists, thickness ≤ material maximum and inside a rate band, part within 3000 × 1500 mm, quantity ≥ 1

Table 10. Formula

Step	Expression
Area	areaM2 = lengthMm × widthMm / 1,000,000
Weight	weightKg = areaM2 × thicknessMm × density (g/cm³)
Material	material = weightKg × material_rate_per_kg
Cutting	cutting = (cutLengthMm / 1000) × cut_rate_per_m
Piercing	piercing = pierces × pierce_rate
Bending	bending = bends × settings.bend_rate_inr
Coating	coating = powderCoat ? areaM2 × 2 × settings.powder_coat_rate_per_sqm : 0
Totals	perPart = sum of the five lines, each rounded half-up to 2 dp; subtotal = round2(perPart × quantity); gst = round2(subtotal × gst% / 100); total = subtotal + gst

Explain (technical notes for the speaker)

- Workflow:
- 1. The calculator form updates a useReducer state; useDebouncedValue waits 350 ms after the last input.
- 2. useEstimate sends POST /api/estimate to the BFF route handler (AbortController cancels stale requests; a request-id guard ignores out-of-order responses).
- 3. The route handler validates the body and forwards POST /api/quotes/estimate to the Spring Core API (3 s timeout).
- 4. Spring bean validation runs; EstimateService loads the material and the rate band (RateRepository.findBand), reads settings (bend_rate_inr, powder_coat_rate_per_sqm, gst_percent) and computes all lines in BigDecimal.
- 5. Spring publishes EstimateCalculated and returns JSON including isSample.
- 6. If Spring is unreachable the handler runs computeEstimate() in TypeScript and sets source = offline; the page shows the badge "Offline estimate (sample)".
- 7. POST /api/quotes additionally persists a quote (quote_no PSL-Q-YYYYMMDD-####) in D5.
- Error cases: HTTP 400 with fieldErrors; 400 "No rate band for this thickness" (for example 3.005 mm).

Slide 31 — M2 — worked example (reference case)

On the slide

- Input: MS, 2 mm, 500 × 300 mm, cut 1600 mm, 4 pierces, 2 bends, powder coat, quantity 10
- Per part INR 248.63; subtotal 2,486.30; GST (18 %) 447.53; total INR 2,933.83
- Same total on Spring, Next.js, PHP and Servlet implementations

Table 11. Reference estimate [V Spring API, 21 Oct 2026]

Line	INR
Material	176.63
Cutting	16.00
Piercing	4.00
Bending	16.00
Coating	36.00
Per part	248.63
Subtotal (× 10)	2,486.30
GST 18 %	447.53
Total	**2,933.83**

Explain (technical notes for the speaker)

- Show the arithmetic: area 0.15 m²; weight 0.15 × 2 × 7.85 = 2.355 kg; material 2.355 × 75.00 = 176.625 → 176.63; cutting 1.6 m × 10.00 = 16.00; piercing 4 × 1.00 = 4.00; bending 2 × 8 = 16.00; coating 0.15 × 2 × 120 = 36.00; per part 248.63.
- The rates (75.00 per kg, 10.00 per m, 1.00 per pierce, 8 per bend, 120 per m², 18 % GST) are SAMPLE values stored in the MySQL database. Say so on the slide.
- Verified: the Spring API returned exactly this JSON on 16 Oct and again on 21 Oct 2026 [V].

Slide 32 — M3 — Enquiry and Drawing Ingestion Module

On the slide

- Description: stores an enquiry (and optional drawing) and returns an id plus a WhatsApp URL
- Accepted files: .dxf, .dwg, .pdf, .png, .jpg, up to 10 MB; content check; random private file names
- Controls: zod validation, prepared statements, rate limit 20 per 15 min, helmet, CORS allow-list, file logs

Explain (technical notes for the speaker)

- Workflow:
- 1. A Next.js server action (or the React Portal, or the PHP form) sends the enquiry: JSON, or multipart/form-data with a drawing field.
- 2. Express applies helmet, the CORS allow-list and the rate limiter (HTTP 429 when exceeded).
- 3. zod validates name, phone, message, optional e-mail, service and machine slugs; invalid input returns HTTP 400.
- 4. multer receives the file; type, size and content are checked; a renamed executable is rejected and no file remains on disk.
- 5. A prepared-statement INSERT writes enquiries and uploads rows (D4); the file is stored under storage/uploads with a random name (D7).
- 6. The API returns 201 {id, whatsappUrl}; request and event logs are appended in the file system.
- 7. Staff users (JWT role admin or staff) list enquiries, change status and download the private file.
- The PHP Estimator Application has its own enquiry form with CSRF token and the same file rules (source = php).

Slide 33 — M4 — Authentication and Access-Control Module

On the slide

- Description: identity, roles and route protection across five applications
- Spring: stateless JWT (HS256, 8 h, claims sub, email, role); Node.js verifies the same token
- Next.js: token in httpOnly cookie, read on the server; Laravel and Servlet: server sessions on the same users table

Explain (technical notes for the speaker)

- Workflow (Spring and Next.js):
- 1. Client posts credentials to the Next.js login route handler.
- 2. The handler calls Spring POST /api/auth/login; Spring checks the bcrypt hash and publishes UserLoggedIn.
- 3. Spring returns a JWT; the handler stores it in an httpOnly, SameSite=Lax cookie (secure in production) and returns only user data.
- 4. Later requests: the Next.js server reads the cookie, verifies the HS256 signature (timing-safe) and forwards Authorization: Bearer to Spring.
- 5. Spring JwtAuthFilter authenticates the principal; SecurityConfig authorises: /api/admin/** needs ADMIN or STAFF; /api/me needs authentication.
- Laravel: login throttled to 5 failed attempts per minute; role middleware; only admin may delete or change estimator settings. Servlet: AuthFilter and session with URL rewriting.

Slide 34 — M5 — Order Tracking Module

On the slide

- Description: read-only lookup of a work order and its status events
- Lookup key: orderNo + phone; wrong phone → 404; phone returned masked
- Rendering: SSR per request, streamed with Suspense; route-level error boundary

Explain (technical notes for the speaker)

- Workflow:

- 1. Client opens /track?orderNo=…&phone=…; the page awaits searchParams (this makes the route dynamic).
- 2. The shell and form stream first; an async server component calls Spring GET /api/work-orders/track with cache: "no-store".
- 3. Spring queries work_orders by order_no and customer_phone, then loads work_order_events in order (D6).
- 4. The server component renders the order card and a seven-stage timeline (order-timeline presenter).
- 5. No match → 404 → "No order matches"; Spring unreachable → built-in demo order marked as demo.
- The Servlet/JSP Desk Application and the React Portal Application implement the same lookup against D6 or the Core API.

Slide 35 — M6 — Work-Order Management Module

On the slide

- Description: staff users create work orders and move them through 7 stages with a history row per change
- Stages: received → cutting → bending → welding → coating → inspection → dispatched
- Three implementations: Spring admin endpoints, Laravel Admin Application, Servlet/JSP staff desk

Explain (technical notes for the speaker)

- Workflow (Spring):
- 1. Staff user sends POST/PUT /api/admin/work-orders (Bearer JWT).
- 2. JwtAuthFilter and SecurityConfig authorise the role.
- 3. WorkOrderService, inside one @Transactional method, saves the work_orders row and inserts a work_order_events row when the status changes.
- 4. After commit, WorkOrderStatusChanged is published (module M7).
- 5. POST /api/admin/work-orders/{id}/events adds a status event with a note.
- Laravel: Work orders screen with "Add status event"; Servlet/JSP: /staff/orders (list, filter, create, update).
- Why events are rows: the timeline is append-only history; the current status is a column for fast filtering.

Slide 36 — M7 — Event Processing and Live Update Module

On the slide

- Description: decouples logging and push from business logic with domain events
- Events: UserLoggedIn, UserRegistered, EstimateCalculated, QuoteSaved, WorkOrderStatusChanged
- Consumers: MongoDB activity_log (all events); STOMP topic /topic/work-orders (status changes)

Explain (technical notes for the speaker)

- Workflow:
- 1. A service publishes a record through ApplicationEventPublisher.
- 2. ActivityEventListener consumes it asynchronously (AsyncConfig); publisher and consumers do not reference each other.
- 3. The listener writes an ActivityLog document in MongoDB (schema-less details per event type).
- 4. For WorkOrderStatusChanged the listener also calls the simple broker convertAndSend("/topic/work-orders", payload).
- 5. The Next.js dashboard (useLiveWorkOrders) subscribes over a STOMP 1.2 WebSocket client; pushed orders are upserted into the query cache. If the socket is unavailable it polls every 15 s.
- Status: socket connection and polling are implemented; an automated end-to-end push test is not part of the verified set (state this).

Slide 37 — M8 — Administration and Rate-Card Management Module (Laravel)

On the slide

- Description: back-office screens for enquiries, machines, services, rate card, work orders and quotes
- Dashboard counts, enquiry status flow new → contacted → quoted → won/lost

- Rate card editing changes estimates without code changes; admin_audit_logs records changes

Explain (technical notes for the speaker)

- Workflow:
- 1. Staff user logs in (session; throttle 5 attempts per minute).
- 2. Role middleware allows admin and staff to view and update; delete and estimator settings require admin (staff receive 403).
- 3. A form post is checked for the CSRF token (missing token → HTTP 419).
- 4. Eloquent writes the shared tables (non-standard table mapping keeps the existing schema).
- 5. Spring reads the rates and settings tables on each estimate request, so a price change takes effect on the next request; the Next.js materials list is cached up to one hour.
- Laravel also provides the Artisan command pslaser:report for enquiry, quote and order totals. 33 routes (php artisan route:list).

Slide 38 — M9 — Customer Portal Module (React Portal Application)

On the slide

- Description: Vite + React SPA with Router and Redux Toolkit
- Routes: /, /machines, /machines/:slug, /estimate, /cart, /login, /register, /my-quotes, /track
- Offline behaviour: a banner "Offline demo data" when APIs are unreachable

Explain (technical notes for the speaker)

- Workflow:
- 1. The browser loads the SPA from Vite.
- 2. Redux Toolkit slices hold the quote cart and the session; React Router maps URLs to pages.
- 3. The estimate page calls the Core API; the enquiry form calls the Enquiry API.
- 4. /my-quotes requires a JWT from Spring login.
- This module demonstrates Unit II topics: components, props, state, events, refs, keys, Router, Flux pattern and Redux.

Slide 39 — M10 — Stand-alone Server-Side Applications (PHP and Servlet/JSP)

On the slide

- PHP Estimator Application: estimator form, enquiry with upload, stats page, .txt/.csv download, cookies, sessions, CSRF token
- Servlet/JSP Desk Application: public /track and /estimate; staff /staff/orders; lifecycle, concurrency and session demos
- Both read/write the MySQL database directly (PDO; JDBC)

Explain (technical notes for the speaker)

- PHP workflow: request → PHP script fills superglobals → CSRF token check → validation → PDO query for rates → compute → render; preferences stored in cookies for 30 days; recent estimates in the session.
- Servlet workflow: Tomcat maps URL → filter chain (AuthFilter protects the /staff pages) → servlet validates and calls a JDBC data-access class → forwards to a JSP view (MVC) → response. Sessions use cookies, with encodeURL for clients without cookies.
- Why both exist: Units I and III require them; both reproduce the reference estimate INR 2,933.83 [V, 14 Oct 2026].

Slide 40 — M11 — Shared Contract and Data Layer

On the slide

- services/CONTRACT.md: schema, JWT claims and secret, price formula, REST shapes, CORS origins
- services/db/schema.sql + seed.sql (seed generated by generate-seed.ts from src/lib/pslaser/data.ts)
- All services read DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD

Explain (technical notes for the speaker)

- This module is the integration mechanism. A change to the formula, a table or a token claim is first made in the contract and then in every service.
- Reset procedure (erases and recreates the database): schema.sql then seed.sql with --default-character-set=utf8mb4. A past import without that option garbled the characters "–" and "·"; this was fixed.

Slide 41 — Module interaction matrix

On the slide

- Which module uses which service and data store

Table 12. Module × component

Module	Next.js	Spring	Node.js	Laravel	PHP	Servlet	MySQL	MongoDB
M1	●	○	○				D2	
M2	●	●			●	●	D2, D3, D5	event
M3	●		●		●		D4	
M4	●	●	●	●		●	D1	event
M5	●	●				●	D6	
M6		●		●		●	D6	event
M7	●	●						D8
M8				●			D2–D6	
M9		●	●				via APIs	

Explain (technical notes for the speaker)

- Use the matrix as a one-slide recap before Results.

Part 8 — Results

Slide 42 — Functional results

On the slide

- Estimate API returns the expected breakdown: per part 248.63, total 2,933.83, isSample true [V]
- Same total on Spring, Next.js, PHP and Servlet [V 14 Oct 2026]
- Estimator shows source badge: "Live estimate (Spring API)" or "Offline estimate (sample)" [V screenshot]
- Tracking returns the seven-stage timeline for PSL-DEMO-001 (status bending) [V screenshot]

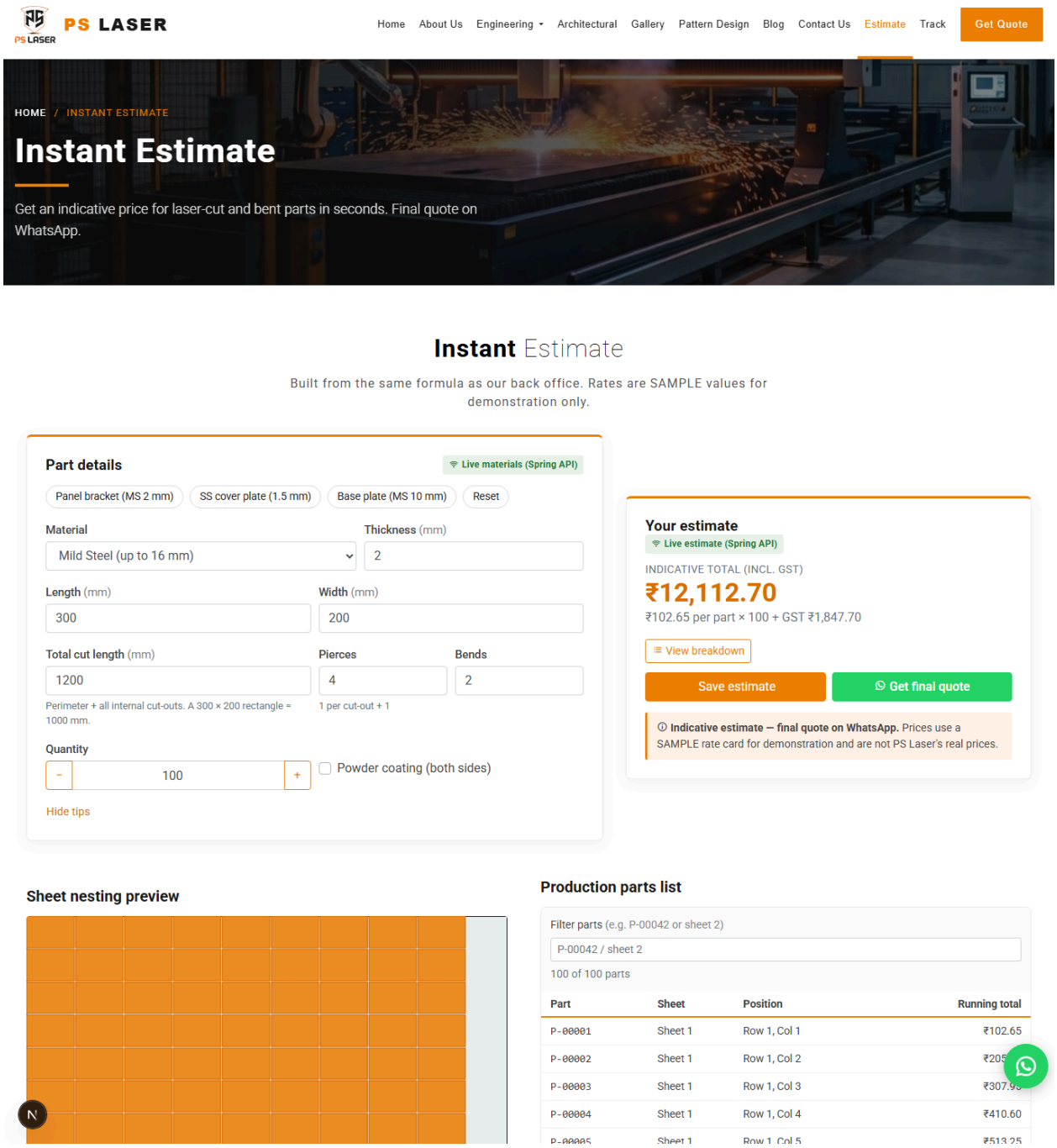


Figure 7. Instant estimate screen (Next.js Web Application, Live estimate)

Explain (technical notes for the speaker)

- Show screenshots 02 and 03 from docs/ppt/screenshots. The black "N" circle at the bottom-left is the Next.js development indicator; crop it out or take new screenshots from a production build.
- Data note: demo orders PSL-DEMO-001 and PSL-DEMO-002 are fake test records.

Visual / file: docs/ppt/screenshots/02-website-quote.png, 03-website-track.png

Slide 43 — Measured results

On the slide

- Estimate API latency (Spring, localhost, 100 sequential requests after 20 warm-up): median 18.6 ms, p95 28.7 ms, max 44.3 ms, min 11.7 ms
- First request after start: 233 ms (cold start)
- Production build (Next.js 16.3.5, Turbopack): compile 12.3 s, type check 10.2 s, 22 static pages in 1.5 s

- Route table: 12 static routes (including /quote with 1 h ISR), 3 SSG blog articles, 6 dynamic routes (/track, /dashboard, 4 API handlers)

Table 13. Measurements [V, 21 Oct 2026]

Metric	Value
Estimate latency median (n = 100)	18.6 ms
p95	28.7 ms
max	44.3 ms
min	11.7 ms
First request (cold)	233 ms
next build: compiled	12.3 s
next build: TypeScript	10.2 s
next build: 22 static pages	1.5 s

Explain (technical notes for the speaker)

- Measurement conditions (say them): 21 Oct 2026, development PC (Intel Core i5-12450H, 15.6 GB RAM, Windows 11), loopback network, a single client, timings taken with curl so they include process start. This is a latency sample, not a load test; no throughput or concurrency figure exists.
- Do not extrapolate to production or to many users.

Slide 44 — Coverage of the syllabus and platform capability

On the slide

- All six units are implemented as features (see Slide 14)
- 8 services responded on their ports [V 14, 16 and 21 Oct 2026]
- Role-based access verified: no token → 401; Laravel staff delete → 403 (automated test); missing CSRF → 303 / 419

Table 14. Objectives versus evidence

Objective	Evidence	Status
O1 identical estimate	INR 2,933.83 on four stacks	[V]
O2 JWT + validated upload	401 without token; Node suite (upload, 429, 400)	[V]
O3 seven stages + history	Laravel test "status event updates work order timeline"; schema	[V]
O4 live push	STOMP socket and polling implemented	[S] end-to-end push not tested
O5 Laravel administration	7 screens return 200 after login; 8 tests pass	[V]
O6 Advanced React	production build, route table, type check	[V]
O7 shared contract	token from Spring accepted by Node.js	[V]
O8 verification	Node 33/33, Laravel 8/8 passed 18 Oct 2026	[V]

Explain (technical notes for the speaker)

- Link results back to the objectives O1–O8 from Slide 7. For each objective, say which result supports it and which part was not tested.

Slide 45 — Screens (gallery slide)

On the slide

- Next.js Web Application: home, /quote, /track, /dashboard login gate
- PHP Estimator Application, Servlet/JSP Desk Application, React Portal Application, Laravel Admin login, Node.js API documentation page

Explain (technical notes for the speaker)

- Use one slide with a 2 × 3 grid of the nine screenshots in docs/ppt/screenshots (01 to 09). Label each with the owning application and port, for example "PHP Estimator Application :8000".

Visual / file: docs/ppt/screenshots/01…09 (captured 21 Oct 2026 from the running system).

Part 9 — Testing (including the software development model)

Slide 46 — Software development model used

On the slide

- Iterative-incremental model with a contract-first (design-first) foundation
- Increment = one module / service; each increment is built, run and verified before the next
- Not waterfall: no single requirements → design → code → test pass; not agile-scrum: no sprints or backlog record exists

Table 15. Phases per increment

Phase	Activity in this project	Artefact
Requirements	Domain facts, syllabus topics, objectives	docs/research/pslaser/brief.md; docs/syllabus/CS3111-units.md
Design	Contract, schema, formula, API shapes	services/CONTRACT.md; services/db/schema.sql
Implementation	One service per increment	services/*, src/
Verification	Unit, integration, cross-stack and QA checks	test suites; handbook section 13
Documentation	Per-unit syllabus mapping and handbook	docs/syllabus/*; docs/handbook/*
Deployment (prepared)	Docker, Caddy, env template	deploy/

Explain (technical notes for the speaker)

- Evidence for the model: (1) services/CONTRACT.md was written before the services; (2) services were built one by one in a fixed order and each has its own README and test command; (3) a QA run on 14 Oct 2026 checked 32 items, and later work re-ran suites (16, 18, 21 Oct) and fixed problems such as UTF-8 garbling and rounding differences; (4) the project was moved to another drive and a deployment kit was added afterwards, which is incremental change.
- Reference: Larman and Basili (2003) describe iterative and incremental development; Royce (1970) is the usual waterfall reference; Boehm (1988) is the spiral model.
- Be transparent: the model was not recorded in a formal plan. Confirm this wording with your team. Also be ready to say that AI-assisted code generation was used, as the course practicals allow, and that you reviewed and verified the output.

Slide 47 — Test strategy

On the slide

- Unit tests: Spring EstimateService with mocked repositories (JUnit 5, Mockito)
- Web-layer tests: Spring MockMvc; integration tests on an in-memory H2 database
- API tests: Node.js node:test + supertest against the real MySQL database; Laravel feature tests (PHPUnit)
- System tests: cross-stack reference estimate; manual QA checklist; Postman collection; scripts/test-estimate.mjs
- Static checks: ESLint, TypeScript compiler, production build (npm run check)

Explain (technical notes for the speaker)

- Black-box tests: HTTP requests with expected status and body (estimate, tracking, auth). White-box tests: service-level unit tests that inspect rounding and band selection.
- Test data hygiene: the Node.js suite deletes every row and file it creates; the Laravel suite uses an in-memory SQLite database and never touches pslaser.

Slide 48 — Test cases executed

On the slide

- Table lists real test cases with expected and actual results

Table 16. Test cases

ID	Test	Input / condition	Expected	Actual
T1	Reference estimate (Spring)	MS 2 mm, 500 × 300, cut 1600, 4 pierces, 2 bends, coat, qty 10	total 2,933.83	total 2,933.83 [V 21 Oct]
T2	Cross-stack equality	same input on Next.js, PHP, Servlet	same total	same total [V 14 Oct]
T3	Thickness over limit	MS 20 mm	HTTP 400	HTTP 400 [V]
T4	Tracking, correct phone	PSL-DEMO-001 + 9000000001	200 + timeline	200, status bending [V]
T5	Tracking, wrong phone	PSL-DEMO-001 + wrong phone	404	404 [V]
T6	Admin endpoint without token	GET /api/admin/enquiries	401	401 (Spring, Node) [V]
T7	Spring login	valid / wrong password	token / 401	token / 401 [V]
T8	Token reuse	Spring token on Node.js /api/enquiries	200	200 [V]
T9	CSRF	PHP bad token; Laravel POST without token	303; 419	303; 419 [V]
T10	Laravel login and 7 screens	admin login	all 200	all 200 [V]
T11	Production build	npm run check	no errors	passed (14 Oct); build re-run 21 Oct [V]

Explain (technical notes for the speaker)

- Read three or four rows aloud (reference estimate, thickness limit, tracking wrong phone, token across services).

Slide 49 — Automated test results and gaps

On the slide

- Node.js: 33 of 33 passed (18 Oct 2026, live MySQL)
- Laravel: 8 of 8 passed, 47 assertions (18 Oct 2026)
- Spring: 15 test methods in source (6 service, 3 web-layer, 6 integration); not re-run for this presentation
- Not tested: responsive visual pass, every image, React Portal → Node.js → database enquiry, end-to-end live push, load and performance

Explain (technical notes for the speaker)

- The first Node.js run on 18 Oct had 9 failures only because MySQL was not running; after starting MySQL all 33 passed. This shows the tests depend on the database being up.
- QA matrix of 14 Oct 2026 (32 items): 21 passed, 9 not tested, 1 failed (disk space on drive C:), 1 blocked (Docker engine stopped).
- Say "not tested" for the gaps; do not describe them as passed.

Slide 50 — Issues found and resolved

On the slide

- UTF-8 garbling ("–", "·") after import without charset → reload with utf8mb4
- One-paisa differences between languages → single rounding rule (round each line, then add)
- localhost resolved to IPv6 for some dev servers → documented 127.0.0.1 workaround
- npm child node process kept ports busy on Windows → documented port-freeing procedure
- Drive C: full → project moved to D:

Explain (technical notes for the speaker)

- These come from docs/handbook (sections 11 and 12). They show the iterative cycle: defect found, cause identified, fix and rule recorded.

Part 10 — Advantages and Disadvantages

Slide 51 — Advantages

On the slide

- Single formula verified across four languages (INR 2,933.83)
- Contract-first design keeps seven services consistent
- Stateless JWT shared by Spring and Node.js; httpOnly cookie in Next.js
- Graceful degradation: offline estimate and demo data labelled as such
- Event-driven logging and live push without coupling business logic
- Role-based administration; price changes need no code change
- Layered, feature-based front end using the Unit V patterns
- Each unit stands alone and is demonstrable

Explain (technical notes for the speaker)

- Tie each advantage to evidence: slide numbers for results, security and architecture.

Slide 52 — Disadvantages and limitations

On the slide

- Rate card is SAMPLE data; estimates are indicative only
- Shared schema duplicates business rules in four estimators
- No machine, DXF analysis, payment, e-mail or SMS integration
- Runs locally; deployment kit prepared but not built or deployed
- Development defaults must be replaced before public use (database credentials, JWT secret)
- Embedded MongoDB loses data on restart; several areas are untested (see Slide 49)
- Eight server processes need a machine with enough RAM; no load test exists

Explain (technical notes for the speaker)

- State limitations plainly; they are part of the engineering result.
- Product and gallery pictures are representative renders; demo orders are fake data.

Part 11 — Conclusion

Slide 53 — Conclusion and future scope

On the slide

- Delivered: contract-driven multi-service platform covering CS3111 Units I–VI
- Verified: identical estimate on four stacks; automated suites passing; production build with SSG, ISR and SSR routes
- Future: real rate card, e-mail/WhatsApp Business notifications, DXF parsing for cut length and pierces, Playwright end-to-end tests, HTTPS deployment (pslaser.online) with managed MySQL and backups

Explain (technical notes for the speaker)

- Close with the honest summary: the system works and is verified where stated; sample rates, missing integrations and the untested areas are the next work.
- Deployment note: a Docker and Caddy kit exists in deploy/; a server and DNS records are still required.

Slide 54 — References

On the slide

- See the reference list below (IEEE format)

Slide 55 — Thank you / Questions

On the slide

- Thank you
- Questions

Explain (technical notes for the speaker)

- Keep the architecture diagram or DFD Level 1 ready as a backup slide for questions.

References

1. R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.
2. G. E. Krasner and S. T. Pope, "A cookbook for using the model-view-controller user interface paradigm in Smalltalk-80," *J. Object-Oriented Program.*, vol. 1, no. 3, pp. 26–49, 1988.
3. M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF, RFC 7519, May 2015.
4. M. Jones, "JSON Web Algorithms (JWA)," IETF, RFC 7518, May 2015.
5. N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annu. Tech. Conf., FREENIX Track*, 1999, pp. 81–91.
6. I. Fette and A. Melnikov, "The WebSocket protocol," IETF, RFC 6455, Dec. 2011.
7. OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/
8. *Systems and Software Engineering — SQuaRE — System and Software Quality Models*, ISO/IEC 25010:2011, 2011.
9. C. Larman and V. R. Basili, "Iterative and incremental developments: A brief history," *Computer*, vol. 36, no. 6, pp. 47–56, Jun. 2003.
10. W. W. Royce, "Managing the development of large software systems," in *Proc. IEEE WESCON*, 1970, pp. 1–9.
11. B. W. Boehm, "A spiral model of software development and enhancement," *Computer*, vol. 21, no. 5, pp. 61–72, May 1988.
12. T. DeMarco, *Structured Analysis and System Specification*. New York, NY, USA: Yourdon Press, 1978.
13. C. Gane and T. Sarson, *Structured Systems Analysis: Tools and Techniques*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1979.
14. S. Newman, *Building Microservices*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2021.
15. C. Richardson, *Microservices Patterns*. Shelter Island, NY, USA: Manning, 2018.
16. W. M. Steen and J. Mazumder, *Laser Material Processing*, 4th ed. London, U.K.: Springer, 2010.
17. *Spring Boot Reference Documentation*, version 3.5. [Online]. Available: https://docs.spring.io/spring-boot/
18. *Next.js Documentation*, Vercel Inc. [Online]. Available: https://nextjs.org/docs
19. *React Documentation*. [Online]. Available: https://react.dev
20. *Laravel Documentation*, version 12.x. [Online]. Available: https://laravel.com/docs/12.x

Part 12 — Additional information for your preparation

12.1 Technical viva questions with answers

Table 17. Question bank

#	Question	Answer (technical)
1	Why seven services instead of one application?	Each CS3111 unit needs its own technology. One business domain implemented seven ways shows each technology on real data; a contract keeps them consistent.
2	How do you guarantee the same estimate in four languages?	One written formula, one rounding rule (round each line half-up to 2 dp, then add) and one reference case (INR 2,933.83) tested on Spring, Next.js, PHP and the Servlet application.
3	Why BigDecimal in Java?	Binary floating point cannot represent decimal money exactly; BigDecimal with HALF_UP gives deterministic two-decimal results.
4	What is a BFF and why use it?	Backend-for-Frontend: Next.js route handlers sit between the browser and the Core API. They validate input, keep the Spring URL and token out of the client bundle, and provide the offline fallback.
5	Why is the JWT in an httpOnly cookie?	JavaScript in the page cannot read it, which limits token theft through XSS. SameSite=Lax limits cross-site sending.
6	How do Spring and Node.js share authentication?	Both use HS256 with the same secret. Spring issues the token (claims sub, email, role, 8 h); Node verifies signature, expiry and role.
7	Difference between SSR, SSG, ISR and CSR in your project?	/track is SSR (per request); most pages are SSG; /quote is ISR with revalidate 3600; the calculator and live board are CSR islands.

#	Question	Answer (technical)
8	Why Zustand and Redux both?	Redux Toolkit is Unit II (React Portal). Zustand is Unit V: small client state, selector subscriptions, persist middleware, no provider boilerplate.
9	What does React.memo do and when is it useless?	It skips re-render when props are shallow-equal. It does not help when props change every render (new function or object identity), which is why useCallback and a memoised input are used with it.
10	useMemo vs useCallback?	useMemo caches a computed value; useCallback caches a function reference. Both depend on dependency arrays.
11	What is useTransition used for here?	Filtering up to 10,000 parts: the input stays responsive and the stale list is dimmed while the filter renders at lower priority.
12	How is real-time update implemented?	Spring publishes WorkOrderStatusChanged after commit; ActivityEventListener sends a STOMP message to /topic/work-orders; the dashboard subscribes over WebSocket; polling every 15 s is the fallback.
13	Why publish events after commit?	If the transaction rolls back, no push or log should describe a change that did not happen.
14	How are uploads secured?	Type allow-list, 10 MB limit, content check (renamed executables are rejected), random private file names, download only for authenticated staff, rate limit.
15	Why does tracking return 404 for a wrong phone?	The response must not reveal whether an order number exists; both values must match.
16	What is CORS and how is it configured?	Browser rule on cross-origin calls. Each API allows an explicit origin list (3001, 5173, 8000, 8001, 8080 in development).
17	Why share a database across services?	Course units must stand alone and share one data model. The cost is duplicated rules; mitigated by the contract and the reference test.
18	What does ddl-auto = validate mean?	Hibernate checks entities against the existing tables at start-up and never creates or alters them, protecting the shared schema.
19	Why work_orders and not jobs?	Laravel owns a jobs queue table; a different name avoids the clash.
20	How does the offline fallback work?	The route handler has a 3 s timeout; on failure it runs the TypeScript copy of the formula and marks source = offline, shown by a badge.
21	What is a discriminated union and where is it used?	A union with a literal "kind" or "status" field that lets TypeScript narrow types, for example RequestState and TrackResult.
22	How do you prevent stale responses in useEstimate?	AbortController cancels the previous request and a request-id guard ignores out-of-order responses.
23	What would change for production?	Real rates, secrets from a secrets manager, HTTPS, managed MySQL with backups, persistent MongoDB, CI with the automated tests. A Docker and Caddy kit is prepared in deploy/.
24	What is not tested?	Responsive visual pass, every image, React Portal → Node.js → database enquiry, end-to-end live push, load and performance.
25	Which model did you follow?	Iterative-incremental with a contract-first foundation; one module per increment, verified before the next.

12.2 Key numbers cheat sheet

Table 18. Numbers to remember

Fact	Value	Evidence
Applications / data stores	7 / 2 (+ disk storage)	[S]
Shared tables	11 (+ Laravel framework tables)	[S]
Machines / services / materials	7 / 8 / 5	[V] queried
Work-order stages	7	[S]
JWT	HS256, 8 h	[S]
Upload limit and types	10 MB; dxf, dwg, pdf, png, jpg	[S]
Enquiry rate limit (default)	20 per 15 min	[S]
Laser bed validation	3000 × 1500 mm	[S]
Reference total	INR 2,933.83	[V]
Estimate latency	median 18.6 ms, p95 28.7 ms (n = 100, localhost)	[V 21 Oct]
next build	12.3 s compile, 22 static pages	[V 21 Oct]
Node.js / Laravel tests	33 / 8 passed	[V 18 Oct]
Spring tests in source	15 methods (not re-run)	[S]
QA matrix (14 Oct)	32 items: 21 pass, 9 not tested, 1 fail, 1 blocked	[V]
Laravel routes	33	[S]
Postman requests	32 (as documented, not re-run)	[S]

12.3 Five-minute demo script (all URLs local)

1. Start MySQL, then Spring (profile dev,mongo-embedded), Node.js, Next.js on 3001; check http://localhost:8081/actuator/health shows UP.
2. Open http://localhost:3001/quote: show the badge "Live estimate (Spring API)". Enter the reference case; result INR 2,933.83 after the 350 ms debounce.
3. Stop Spring; repeat: the badge changes to "Offline estimate (sample)" with the same result (TypeScript fallback). Restart Spring.
4. Open /track?orderNo=PSL-DEMO-001&phone=9000000001: seven-stage timeline, status bending. Change the phone number: no match (404 path).
5. Open Laravel http://localhost:8001/login, sign in with the local admin (credentials in services/db/SEED_CREDENTIALS.md; do not show the password), add a status event to PSL-DEMO-001, then refresh /track.
6. Open http://localhost:3001/dashboard: login gate, then the live board badge (STOMP or polling).
7. Run npm test in services/node-api (33 tests) and php artisan test in services/laravel-admin (8 tests).

12.4 Do not claim (keep the presentation accurate)

- Do not say the platform is deployed or live on pslaser.online.
- Do not present estimates as real PS Laser prices; rates are sample data.
- Do not claim performance or load results beyond the single-client latency sample.
- Do not say the live push was tested end to end; the socket and polling are implemented.
- Do not say the Spring tests were run today; 15 test methods exist but were not re-run.
- Do not claim the machines are integrated with the software.
- Do not claim novelty in cost estimation; the contribution is the integrated, consistent multi-service design.
- Do not show the admin password or the JWT secret on screen.

12.5 Slide design advice (match the Unit V deck)

- Reuse the Unit V frame: white slide with rounded border, orange (#D34817-style) title band on the first slide, grey title text, orange bullets, date in the footer (21-Oct-26 style).
- Use 4:3 like the Unit V deck, or 16:9 if the template allows; keep one idea per slide and at most 6 bullets.
- Put code, file paths and long tables in the speaker notes; keep the diagram as the visual.
- Use the colour code of the DFDs consistently: black data flow, blue data-store access, red dashed domain event, green external link.

12.6 Fifteen-minute short path

If you have 15 minutes, present these 20 slides: 1 (title), 3 (introduction), 5 (problem), 7 (objectives), 8 (literature), 10 (technology stack), 15 (architecture), 17 (DFD Level 0), 18 (DFD Level 1), 20 (DFD Level 2 estimate), 23 (Unit V layered architecture), 28 (module overview), 30 (M2 estimation), 31 (worked example), 43 (measured results), 46 (development model), 49 (automated tests and gaps), 51 (advantages), 52 (limitations), 53 (conclusion). Keep the rest as backup. Slide numbers refer to the sequence in this document; renumber after you build the deck.

12.7 Image and file index

Table 19. Files for the slides

File	Use on slide
docs/ppt/_build/dfd-level0.png	DFD Level 0
docs/ppt/_build/dfd-level1.png	DFD Level 1
docs/ppt/_build/dfd-level2-estimate.png	DFD Level 2 (2.0)
docs/ppt/_build/dfd-level2-orders.png	DFD Level 2 (order lifecycle)
docs/ppt/_build/unit5-layers.png	Unit V layered architecture
docs/submission/PSLaser_Architecture_Diagram_detailed.png	System architecture
docs/ppt/screenshots/01…09 .png	Results and screens gallery

File	Use on slide
docs/submission/PSLaser_FINAL_IEEE_Research_Paper.pdf	Backup: full paper